

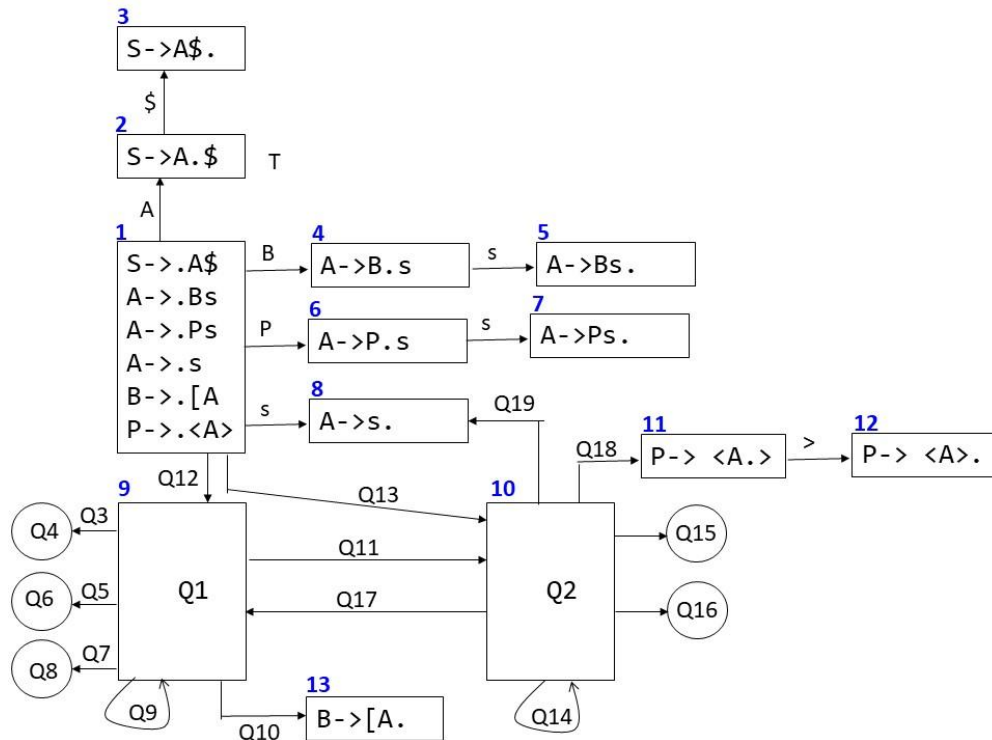
1. Consider the Language L_1 defined by the grammar G_1 :

(4 points)

1. $S \rightarrow A\$$
2. $A \rightarrow Bs$
3. $A \rightarrow Ps$
4. $A \rightarrow s$
5. $B \rightarrow [A$
6. $P \rightarrow <A>$

a. Is G_1 LL(1)? Why or Why not? (0.25 points)

b. A partial LR(0) CFSM is shown below for G_1 . Complete the CFSM by filling in Q1-Q19 i.e., you need to fill states 9 and 10, mark the edges, and fill in the circles. The circles contain the state that the edge connects to (state numbers are shown outside the box at the top-left corner). (3.75 points)



2. Consider the FPE expression $a := (((b*c)/d)+e)$. Draw the AST for the expression.

(1 point)

3. Perform Common Subexpression Elimination (CSE) on the sequence shown on the right. **(5 points)**
- For each statement, after executing that statement, show available and killed expression(s) (2.5 points)
 - Suppose E and C were aliased in the sequence shown. How will that change the results of CSE? (1 point)
 - Suppose B and D were aliased in the sequence shown. How will that change the results of CSE? (1 point)
 - Describe the effect that aliasing has on CSE w.r.t. number of sub expressions and code redundancy. Your answers must take into account observations from your answers to parts b and c. (0.5 points)
- A = 7;
 - B = A + 2;
 - C = A + B;
 - D = C + B;
 - B = C + B;
 - A = A + B;
 - E = C + D;
 - F = C + D;
 - G = C + B;
 - H = E + F;
4. For the sequence shown in the previous question (above), perform register allocation with 3 registers. When choosing registers to allocate, always allocate the lowest-numbered register available. When choosing registers to spill, choose the register holding a value that will be used farthest in the future (in case of a tie, choose the lowest-numbered register).
- Show what variables are live at each statement (assume no variables are live out of statement 10) **(1 point)**
 - For each statement, show register-variable mapping after performing bottom-up register allocation. You must use the method discussed in class. Mark dirty registers with a '*'. assignment. **(5 points)**
 - For each statement, show code generated. **(5 points)**
5. Assume that a machine has 2 ALUs, a MU and a LD/ST unit. The architecture has five instructions: ADD, SUB, MUL, LD and ST. ADD and ST take one cycle each, SUB, MUL and LD take two cycles each. The reservation tables for the instructions are shown in the table (note that the MUL is fully pipelined, so a new MUL instruction can be issued in every cycle). For the code snippet shown in the table, fill in the dependencies for each statement, also, fill in the minimum number of cycles that it takes for this code to execute. Give a schedule of instructions that are executed in each cycle if one were to use *list scheduling* **(4 points)**

Reservation tables										Code												
ADD	ALU0	ALU1	MU	LD/ST	ALU0	ALU1	MU	LD/ST	MUL:	1: LD A, R1; 2: LD B, R2; 3: R3 = R1 + R2; 4: LD C, R4 5: R5 = R1 * R3; 6: R6 = R3 * R2; 7: R7 = R1 - R5; 8: R8 = R4 + R6; 9: R9 = R7 + R8 10: ST(R9), D;												
	X					X					<table><tr><td>ALU0</td><td>ALU1</td><td>MU</td><td>LD/ST</td></tr><tr><td></td><td></td><td>X</td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr></table>	ALU0	ALU1	MU	LD/ST			X				
ALU0	ALU1	MU	LD/ST																			
		X																				
SUB	ALU0	ALU1	MU	LD/ST	ALU0	ALU1	MU	LD/ST	ST:													
	X					X				<table><tr><td>ALU0</td><td>ALU1</td><td>MU</td><td>LD/ST</td></tr><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td></tr></table>	ALU0	ALU1	MU	LD/ST								
	ALU0	ALU1	MU	LD/ST																		
X						X																
LD	ALU0	ALU1	MU	LD/ST	ALU0	ALU1	MU	LD/ST	<table><tr><td>ALU0</td><td>ALU1</td><td>MU</td><td>LD/ST</td></tr><tr><td></td><td></td><td></td><td>X</td></tr><tr><td></td><td></td><td></td><td></td></tr></table>	ALU0	ALU1	MU	LD/ST				X					
	ALU0	ALU1	MU	LD/ST																		
				X																		
X						X																
				X				X														